

LinkedIn Documentation Guide

Dual-MCU IoT Sensor Node Project

Build-in-Public Strategy · Milestone Showcase Posts · Profile Optimisation

Targeting Embedded Systems Internships + Full-Time Roles · Final Year ECE Student

LinkedIn Guide v1.0

The Core Philosophy

LinkedIn for an ECE student is not a resume photocopy. It is the only public channel where a hiring engineer can watch you think. This project gives you 14 months of genuine technical content — things most candidates cannot produce because they never built anything this deep. Your job is to document it honestly, consistently, and specifically.

Two modes run in parallel throughout the project: **Build-in-Public posts** every 3–4 weeks showing real progress (including failures), and **Milestone Showcase posts** at each GitHub tag, written for a hiring audience. Neither should sound like a product launch. Both should sound like an engineer who genuinely knows what they built.

Step 0 — Profile Before You Post

No post performs well if the profile it leads back to looks incomplete. Fix these once, then never touch them again until graduation.

Section	What to write	Common mistake
Headline	ECE Student · Embedded Systems · STM32 · FreeRTOS · IoT — be specific, not generic	"Engineering Student at XYZ University" — says nothing
About	3 sentences: what you build, what technologies you use, what you're targeting. No buzzwords. Link to GitHub.	Walls of adjectives ("passionate", "driven") with no technical content
Projects	Add this project immediately. Title: "Dual-MCU IoT Sensor Node (STM32F411 + ESP32)". Update description at each milestone.	Projects section empty or showing only college coursework
Skills	Add as you build: Embedded C, STM32, FreeRTOS, UART, I2C, SPI, ESP-IDF, MQTT, Git, KiCad. Only add what you can explain.	Listing "Python" and "Java" instead of embedded-specific skills
GitHub link	Pin this project repo on your GitHub profile. Keep README current.	GitHub profile with no pinned repos or empty READMEs

The Two Post Types

Type 1 — Build-in-Public Post

Frequency: every 3–4 weeks, roughly once per major subphase completed. These are short, honest, specific posts about what you built, what broke, and what you learned. They build an audience of peers and engineers who appreciate genuine engineering content.

■ What makes a build-in-public post work

What it is: Specificity. 'I implemented a custom UART protocol with CRC-16 between STM32 and ESP32' is 10× more credible than 'I worked on embedded communications this week.' The reader should learn something real from every post, even if they don't follow up.

Why it's here: You are building an audience of embedded engineers and hiring managers, not general followers. They can immediately tell if you are being vague because you don't actually understand what you built. Specificity signals competence.

Build-in-Public Post Structure (150–250 words):

- **Line 1 (hook):** One specific, concrete sentence. Not 'I learned a lot.' Example: 'I spent 3 days debugging an I2C bus lockup on STM32F4 — here's what was actually happening.'
- **Lines 2–6 (the substance):** What you built or debugged. One specific technical detail. What the UART dump looked like, what the register value was, what the reference manual said. Real engineers find this compelling.
- **Lines 7–9 (what you learned):** One genuine insight. Not 'I learned that embedded is hard.' Example: 'The STM32F4 I2C peripheral cannot self-recover from a NACK lockup — you must bit-bang 9 SCL pulses via GPIO to release the bus.'
- **Last line:** Where you're going next. Pulls people into the ongoing series.
- **Image:** UART terminal output, oscilloscope/logic analyser screenshot, or a photo of the hardware. Always attach an image — posts with images get 3× more reach.
- **Hashtags (5 max):** #EmbeddedSystems #STM32 #FirmwareDevelopment #IoT #ECE

Type 2 — Milestone Showcase Post

Frequency: once per GitHub tag (9 total across the project). These are longer, more polished posts that explain the full milestone — what the system can now do that it couldn't before. Written for a hiring engineer who sees it in their feed.

Milestone Showcase Post Structure (250–400 words):

- **Title line:** 'Just tagged v0.1-peripheral-drivers — here's what it took.' Simple, specific, uses the actual tag name.
- **What the system can do now:** 2–3 sentences describing the new capability. What the system could NOT do before this milestone. What it CAN do now.
- **The hardest technical problem:** One paragraph on the most difficult thing you solved. Specific registers, specific error messages, specific fix. This is what separates your post from a project description.
- **What you would do differently:** One honest sentence. This shows maturity and that you understand your own work critically — a rare quality in a student.
- **GitHub link + tag:** Link directly to the tagged commit. Example:
github.com/yourname/dual-mcu-iot-node/tree/v0.1-peripheral-drivers
- **YouTube link** (if video is live): 'Full walkthrough on YouTube — link in comments.'
- **Image:** The most visually striking artifact from this milestone — logic analyser capture, SSD1306 showing live data, terminal output.
- **Hashtags:** #EmbeddedSystems #STM32 #FirmwareDevelopment #BareMetalProgramming #IoT

Complete 14-Month Post Plan

Every post below has a type (BIP = build-in-public, MS = milestone showcase), a title formula, and the one technical fact that makes it credible. You know which fact to use because you built it.

Phase	Type	When	Title Formula
Phase 0	BIP	Week 1–2	"I replaced Arduino IDE with ARM GCC + Make + OpenOCD on Fedora. Here's why and how."
Phase 0	MS	Week 3	"Tagged v0.0-toolchain-verified — bare-metal STM32 from first principles."
Phase 1	BIP	Week 5–6	"Wrote a UART driver from scratch — 47 lines that replaced 500 lines of HAL."
Phase 1	BIP	Week 8–9	"I2C bus lockup on STM32F4 — 3 days to find an errata that ST buried in the manual."
Phase 1	MS	Week 10	"Tagged v0.1-peripheral-drivers — register-level UART, I2C, SPI, Timer IC, DHT22."
Phase 2	BIP	Week 13–14	"Ported FreeRTOS to STM32 by hand — no CubeMX, no code generation."
Phase 2	BIP	Week 16	"ESP32 + ESP-IDF from scratch — why I didn't use Arduino for this project."
Phase 2	MS	Week 17	"Tagged v0.2 — FreeRTOS 6-task architecture + ESP32 WiFi/MQTT baseline."
Phase 3	BIP	Week 20	"Designed a binary UART protocol from scratch — frame format, CRC-16, state machine."
Phase 3	MS	Week 21	"Tagged v0.3-inter-mcu-protocol — custom framed protocol, validated end-to-end."
Phase 4	BIP	Week 25	"Wrote a custom STM32 bootloader from scratch — flash layout, linker scripts, jump sequence."
Phase 4	MS	Week 30	"Tagged v0.4-ota-pipeline — OTA firmware update over WiFi/MQTT with zero physical access."
Phase 5	MS	Week 35	"Tagged v1.0-system-integration — 72-hour soak, watchdog, full system stable."
Phase 6	BIP	Week 40	"Translating a dev-board prototype to a custom PCB in KiCad — what changes, what doesn't."
Phase 6	MS	Week 42	"Tagged v1.1-pcb-bringup — custom PCB designed, fabricated, OTA-updated."

Post Templates with Technical Anchors

Each template below includes the specific technical fact that makes it credible. Replace the [brackets] with your actual numbers and observations.

Build-in-Public Template — Debugging Post

3 days debugging an I2C bus lockup on STM32F4. Here's exactly what was happening. The STM32F411's I2C peripheral has a known errata: after a NACK during a transaction, it sets the BUSY flag in I2C_SR2 and never clears it. The peripheral is now effectively dead — no amount of software reset fixes it. The fix: when the BUSY flag is set with no active transaction, switch SCL and SDA to GPIO output mode and manually clock SCL 9 times. This forces the slave to complete its current byte and release the bus. Then send a manual STOP condition before re-initialising the I2C peripheral from scratch. What the SR1 register looked like before recovery: 0x[XX]. What it looked like after: 0x0000. The reference manual mentions this requires "bus recovery" in one sentence in Section 27.3.4. That's it. No example code. Now have this running in production in the SensorAcquisition FreeRTOS task before every I2C transaction. Zero lockups in 48 hours of testing. Next: implementing the full FreeRTOS task architecture — 6 tasks, 3 queues. #EmbeddedSystems #STM32 #FirmwareDevelopment #IoT #ECE

Milestone Showcase Template — OTA Pipeline

Just tagged v0.4-ota-pipeline. What I can do now that I couldn't do 8 weeks ago: update the STM32F411 firmware wirelessly, from my laptop, with no USB cable, no ST-Link, no physical access to the hardware. The pipeline: publish a .bin file to an MQTT topic on my laptop → ESP32 receives it in 128-byte chunks → forwards each chunk over a custom UART protocol to STM32 → STM32 bootloader writes chunks to flash sector by sector → computes CRC-32 over the full received image → if it matches, sets the OTA flag and reboots → bootloader picks up the new firmware on next boot. The hardest part: the linker script. The application binary must be compiled with FLASH ORIGIN = 0x08010000, not 0x08000000. Getting this wrong produces a HardFault on the first instruction after the bootloader jumps — not an obvious error when you don't know where to look. What I'd do differently: I'd design the OTA chunk protocol to include a chunk index from the start, rather than adding it after discovering dropped chunks mid-transfer. Full code at: [github.com/\[yourname\]/dual-mcu-iot-node/tree/v0.4-ota-pipeline](https://github.com/[yourname]/dual-mcu-iot-node/tree/v0.4-ota-pipeline) Walkthrough video: [YouTube link] #EmbeddedSystems #STM32 #ESP32 #FirmwareDevelopment #OTA

Final Project Showcase Template (at v1.1-pcb-bringup)

14 months ago I started this with intermediate C knowledge and no idea what a linker script was. Today I tagged v1.1-pcb-bringup. What the project is: a dual-MCU IoT sensor node. STM32F411 Black Pill handles all sensor acquisition and firmware logic. ESP32 handles WiFi and MQTT. The two communicate over a custom framed UART protocol with CRC-16 error detection. Firmware updates happen over WiFi via MQTT — no physical access to the hardware required. What I built from scratch: → Register-level peripheral drivers (UART, I2C, SPI, Timer Input Capture, GPIO) — no HAL → FreeRTOS 6-task architecture with queues and priority-based scheduling → Custom binary UART protocol with CRC-16, byte stuffing, ACK/NAK, retransmission → Custom bootloader with flash write, CRC validation, and application jump → End-to-end OTA firmware update pipeline → KiCad PCB — fabricated and operational What I learned that isn't in any course: that every abstraction layer you skip forces you to understand the layer below it. Every bug at the register level taught me something a HAL would have hidden. GitHub: [github.com/\[yourname\]/dual-mcu-iot-node](https://github.com/[yourname]/dual-mcu-iot-node) YouTube series: [channel link] #EmbeddedSystems #STM32 #ESP32 #FirmwareDevelopment #PCBDesign #IoT #ECE #Hiring

What Not to Do

Anti-pattern	Why it fails	What to do instead
'Excited to share that I'm working on an embedded project!'	No content. No reason to read further. Zero credibility signal.	Share the specific technical thing you just figured out — not that you're working on it.
Posting a screenshot of your IDE with no explanation	Code screenshots are unreadable and say nothing on their own.	Show terminal output, UART logs, logic analyser captures — things that prove the system works.
'This project taught me so much about embedded systems!'	Every student says this. It is meaningless without specifics.	State the one specific thing you learned. 'I learned that BSY in I2C_SR2 never clears after a NACK.'
Posting only when things go well	Looks curated. Engineering is mostly debugging. Hiring engineers know this and distrust clean-success narratives.	Post specifically about what broke and how you diagnosed it. That content is rare and valuable.
Posting too frequently (daily)	Dilutes quality. Turns your feed into noise. People unfollow — or worse, stop reading.	Once per major subphase or milestone. Quality beats frequency every time.
Generic hashtags (#coding #tech #student)	Reaches wrong audience. Embedded engineers search specific tags.	#EmbeddedSystems #STM32 #FirmwareDevelopment #BareMetalProgramming #IoT — be specific.
Not linking GitHub	Interested engineers have no way to verify the claim.	Every technical post links to the specific tagged commit. Credibility is the whole point.

Connection Strategy

LinkedIn reach compounds. Each post that does well introduces you to new engineers who then see your next post. Build the network before you need it.

- **Connect with embedded engineers at companies you want to work at.** Send a connection request with a note: 'I'm building a dual-MCU IoT node from scratch — your work on [specific thing] at [company] is directly relevant to what I'm learning. Would appreciate the connection.' Specific = gets accepted. Generic = gets ignored.
- **Engage genuinely on other embedded engineers' posts.** A specific technical comment ('This matches what I saw with STM32F4 I2C — did you also hit the NACK lockup errata?') is worth 100 generic likes. Engineers notice people who understand what they're talking about.
- **Tag relevant people when you cite their content.** If a blog post or GitHub repo helped you solve a problem, tag the author. Most embedded engineers appreciate genuine acknowledgement and will engage.
- **Connect with your university's alumni in embedded roles.** Alumni are statistically more likely to respond than cold contacts. Search LinkedIn for '[Your University] + Embedded' and connect with a specific note.
- **Don't mass-connect with recruiters.** Recruiter connections dilute your feed and give you zero technical credibility signal. Let hiring engineers find you through your content — that's a far stronger signal.

Targeting Embedded Internships and Full-Time Roles

This project demonstrates skills that are genuinely scarce in new-graduate candidates. Most ECE graduates have done Arduino and some Raspberry Pi. You are one of a small number who can say:

- I wrote register-level peripheral drivers without HAL — I know what the RCC clock enable actually does
- I designed and implemented a custom framed UART protocol with CRC error detection from scratch
- I wrote a custom bootloader that validates firmware integrity before jumping to the application
- I implemented an end-to-end OTA update pipeline over MQTT — the firmware updates wirelessly
- I ran a 72-hour soak test and debugged the integration failures that appeared only under load

How to frame this for specific roles:

Role	What to emphasise	What to downplay
Embedded Firmware Internship	Register-level drivers, HardFault debugging, FreeRTOS task design. Show RM0383 sections you used. Show the ring buffer implementation.	PCB design and cloud/MQTT aspects — save those for the job posting if it mentions them.
IoT / Connected Devices Role	OTA pipeline, MQTT protocol, ESP32 integration, end-to-end system reliability. Emphasise the 72-hour soak and the watchdog recovery.	Low-level driver details — lead with the system capability, not the register bits.
RTOS / Safety-Critical	FreeRTOS task architecture, priority design, mutex vs semaphore reasoning, watchdog configuration. Show the ISR-safe API discipline.	Web/cloud aspects. Focus on determinism and reliability.
Hardware / Mixed-Signal	PCB design in KiCad, decoupling strategy, level-shifting for HC-SR04, schematic capture. Show the Gerber files and fabricated board.	Pure firmware aspects — let the hardware artifacts speak.

By the time you tag v1.1-pcb-bringup, your LinkedIn profile will show 14 months of documented engineering work — 15 posts, 9 GitHub tags, a YouTube series, and a fabricated PCB. That is a portfolio that requires no explanation.